Fuzzy Logic Car-Following Controller

Simulink model with a Mamdani fuzzy controller

Computational Intelligence course project, January 2018, MATLAB R2017b

Authors: Hossein Mosaffa, Seyed Hesam Salehnejad, Ali Nikhalat Jahromi

Supervisor: Dr. Abdollahi
Amirkabir University of Technology, Faculty of Electrical Engineering, Tehran, Iran

1. Overview

This project keeps one car behind another at a fixed gap. A lead car follows a speed command, and a follower car has to stay 25 units behind it. The follower is steered by a fuzzy logic controller that looks at two numbers: how far the current gap is from 25, and the speed difference between the two cars, which is exactly how fast the gap is changing.

Everything is built in Simulink with the Fuzzy Logic Toolbox. The original course report and slides were written in Persian and describe the same design. This document summarizes the project in English, working from the model files themselves (the Simulink model and the Rule.fis controller), so the numbers below come from those files.

2. How the model is put together

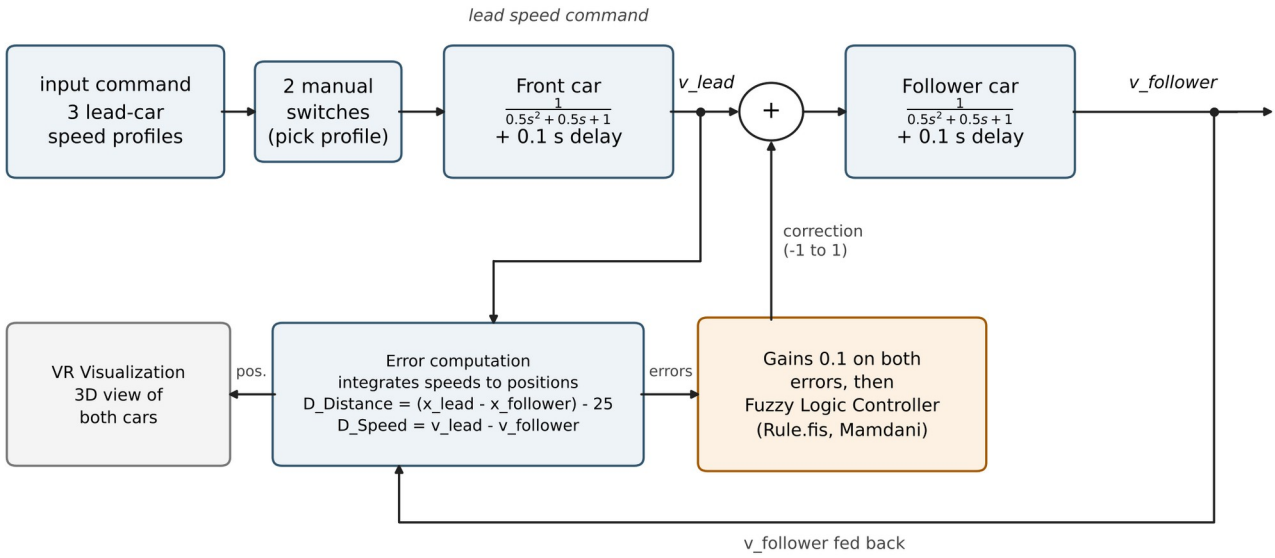


Figure 1. Signal flow of the Simulink model, redrawn from the saved model file.

Input command. A subsystem produces three alternative speed profiles for the lead car, and two manual switches pick the one in use. The profiles are described in section 5.

Cars. Both cars use the same dynamics, a second-order transfer function followed by a 0.1 s transport delay. The delay stands in for driver reaction time and vehicle lag. The input to each car is treated as a speed command, and the output is its actual speed.

Error computation. A subsystem integrates both speeds into positions. The lead car's integrator starts at 5 and the follower's at 0, so the cars begin 5 units apart. The subsystem outputs D_Distance, which is the current gap minus the desired gap of 25, and D_Speed, which is lead speed minus follower speed. It also passes both positions on for the 3D view.

Controller. Both errors are multiplied by 0.1 and fed to a Fuzzy Logic Controller block that loads Rule.fis. Its output is added to the lead car's speed, and that sum is the command for the follower car.

Visualization. A VR Sink block (Simulink 3D Animation) draws both cars. It only displays the result and has no effect on the control loop.

The simulation runs from 0 to 600 with a variable-step solver chosen automatically.

3. Car dynamics

Each car is modelled as $1 / (0.5\,s^2 + 0.5\,s + 1)$. That corresponds to a natural frequency of about 1.41 rad/s and a damping ratio of about 0.35, so the speed overshoots its command. A unit step produces a peak of roughly 1.3, about 30 percent overshoot, near 3.5 s, followed by a few decaying oscillations (figure 2). The transport delay only shifts this curve by 0.1 s.

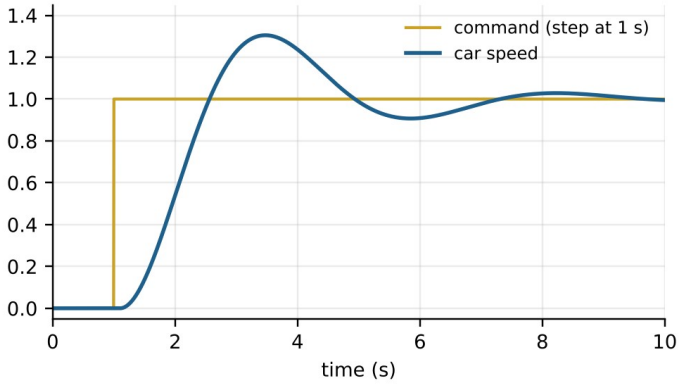


Figure 2. Speed response of one car to a unit step at t = 1 s, computed from the transfer function and delay.

4. The fuzzy controller

The controller is a Mamdani system with two inputs and one output, defined in Rule.fis. It uses min for AND and for implication, max for aggregation, and centroid defuzzification. All three variables span -1 to 1. The output variable is named signal-improvment in the file (spelling as saved).

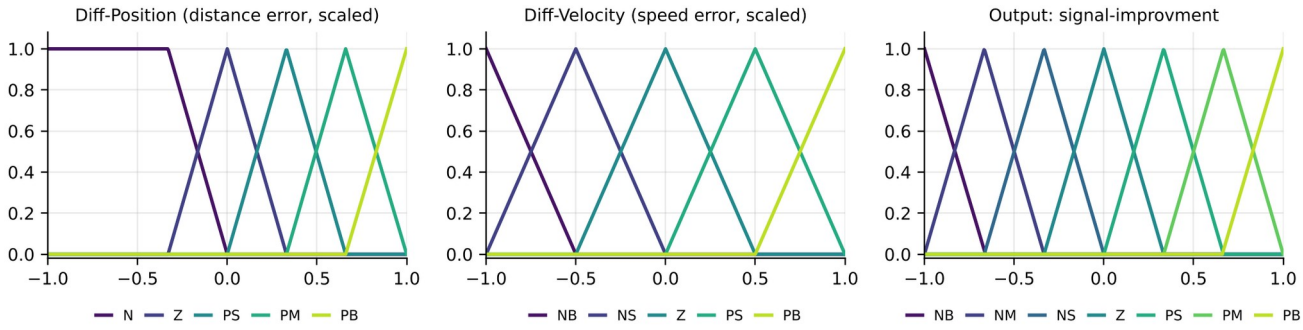


Figure 3. Membership functions for the two inputs and the output, plotted from Rule.fis.

The 25 rules cover every combination of the five distance-error sets and the five speed-error sets. The table lists the output set chosen for each pair.

Diff-Position \ Diff-Velocity	NB	NS	Z	PS	PB
N	NB	NB	NB	NB	NB
Z	NM	NS	Z	PS	PM
PS	NS	Z	PS	PM	PB
PM	Z	PS	PM	PB	PB
PB	PS	PM	PB	PB	PB

Table 1. Rule base. Rows are the distance-error set, columns the speed-error set, entries the output set.

A negative distance error means the follower is closer than 25 units. In that row every rule returns NB, so the controller brakes as hard as it can whatever the speed difference. In the other rows the output grows with both errors: the further behind the follower is, and the faster the lead car is pulling away, the stronger the push to speed up. With both errors at zero the output is zero.

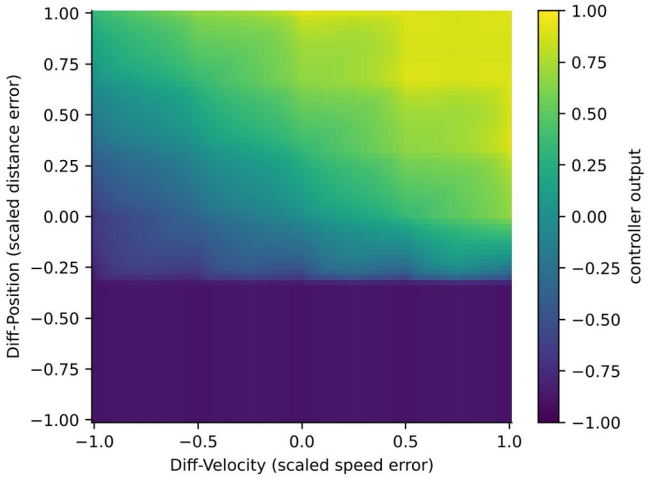


Figure 4. Control surface of the fuzzy controller, computed by evaluating Rule.fis over a grid of both inputs.

5. Lead-car speed profiles

The lead car is driven by one of three commands, built by summing Step, Ramp and Sine Wave blocks. The model does not state physical units, so values are given as plain numbers and times in seconds.

Profile	Blocks used	Resulting command
1	Steps to 30 at t=0, then +20 at 100, -10 at 200, -5 at 300; sine of amplitude 5 at 0.1 rad/s	Levels 30, 50, 40, 35 with a sine ripple of 5 on top
2	Ramps with slope +1 from t=0 and -1 from t=200; steps +10 at 100, -10 at 300, -20 at 400	Climbs to 210 by t=200, then steps down to 200 and 180
3	Ramps with slope +2 from t=0, -2 from t=100, -1 from t=150, +1 from t=200	Climbs to 200 by t=100, holds, falls to 150 by t=200 and stays

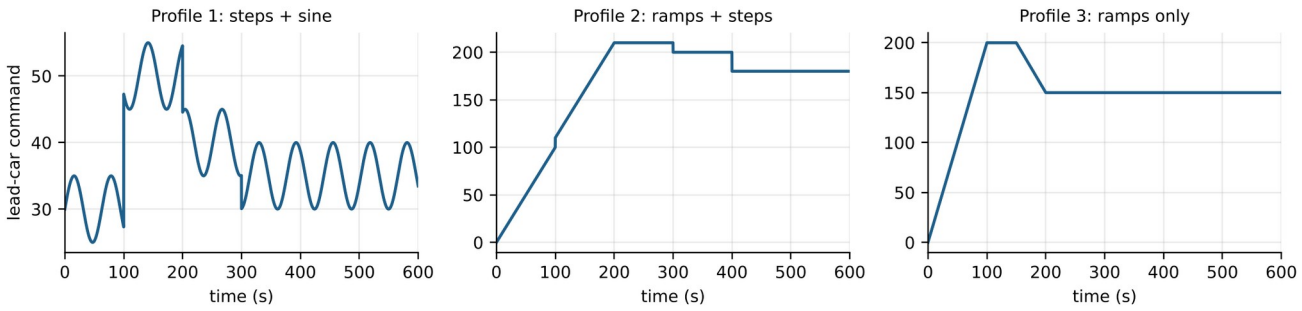


Figure 5. The three lead-car commands, computed from the block parameters in the model file.

6. Files and running the model

File	What it is
`T_C.slx`	The Simulink model.
`Rule.fis`	The fuzzy inference system (inputs, output, membership functions, 25 rules).
`README.md`	Explanation of the model and the rule base.
`Project_Summary_Report.pdf`	This document.

Open T_C.slx in MATLAB. The model was saved in R2017b and needs Simulink and the Fuzzy Logic Toolbox. Keep Rule.fis in the working folder, or load it into the workspace with readfis, so the controller block can find it. The VR block needs Simulink 3D Animation and a world file named octavia_scene_lchg_2cars.wrl, which is not part of this project. Without them, remove or comment out the VR block and the rest of the model still has everything it needs. The Speed scope shows the lead command, lead speed and follower speed.

7. Things worth knowing

The two input gains are 0.1 in the saved model, while a model screenshot in the original slides shows 1, so the slides probably come from an earlier version. The controller's output is limited to the range -1 to 1 and is added to the lead car's speed, so it acts as a small trim around that speed rather than a full speed command. The project files contain no saved closed-loop results, which is why this report shows none. Running the model with the Speed scope open will produce them.